Federated Learning in SDVs: Optimizing Communication Frequency

Spyridon Arvanitis
Department of Informatics and Telematics
Harokopio University of Athens
Athens, Greece

***Abstract*—Deploying Federated Learning (FL) on Software-Defined Vehicles (SDVs) involves balancing wireless transmission overhead, client model divergence, and onboard compute resource contention with safety-critical vehicle workloads. Conventional vehicular FL frameworks typically assume dedicated onboard compute and rely on static local training intervals, which can lead to uplink channel congestion and compute starvation for co-located real-time services. In this paper, we propose a runtime heuristic policy engine that dynamically regulates local training epochs and client upload schedules based on available compute slack, instantaneous link throughput, and observed local loss progress. We evaluate the approach on a non-IID partitioned CIFAR-10 benchmark across realistic V2X channel conditions. The proposed policy matches the validation loss trajectory of high-frequency synchronization baselines while lowering cumulative uplink bandwidth consumption by more than 70% and minimizing transmission buffer drops.**

***Index Terms*—Federated Learning, Software-Defined Vehicles, V2X Networks, Resource Allocation, Mixed-Criticality Systems.**

I. Introduction

Modern Software-Defined Vehicles (SDVs) operate as mobile computing platforms running machine learning workloads for driver assistance, cabin perception, and vehicular diagnostics. Since these systems continuously generate substantial volumes of high-dimensional sensor data, Federated Learning (FL) provides a practical paradigm for training shared models across fleets without transferring raw telemetry to cloud infrastructure.

However, executing FL in vehicular networks introduces operational constraints that differ from conventional edge environments. First, Vehicle-to-Everything (V2X) wireless links exhibit time-varying channel attenuation, interference, and limited capacity. Synchronizing high-dimensional parameter updates across multiple vehicles during each communication round can quickly congest local cellular infrastructure. While reducing synchronization frequency mitigates network contention, extended local training on heterogeneous (non-IID) vehicular data induces model divergence and degrades global convergence.

Second, SDVs rely on consolidated, virtualized hardware architectures where FL workloads must share CPU and accelerator capacity with safety-critical vehicle services (e.g., ADAS tracking and path planning). Unlike general edge devices where training operates as a primary workload, an in-vehicle FL client must yield execution slack whenever safety-critical processes demand compute. Enforcing static training schedules without accounting for co-located processor occupancy risks resource contention with higher-priority tasks.

Existing vehicular FL scheduling solutions generally focus on link-level optimization, rate adaptation, or joint transmit-power allocation, while treating onboard compute as either fixed or dedicated. To address this gap, we present a deterministic runtime policy engine that coordinates compute allocation, local training duration, and upload decisions. The engine monitors co-located workload occupancy, channel capacity, and model divergence to transition clients between active synchronization, continued local training, and idle states.

The main contributions of this work are:

- We formulate the vehicular FL scheduling problem by jointly accounting for uplink bandwidth constraints, client model drift, and safety-critical compute contention.
- We develop an online, deterministic heuristic engine that evaluates the marginal utility of local training iterations and dynamically triggers model upload, local training, or process suspension.
- We validate the proposed mechanism against static local-epoch baselines on a non-IID vehicular testbed, demonstrating substantial communication savings and fewer packet drops under bandwidth limits.

II. Related Work

Vehicular federated learning literature can be broadly categorized into three primary domains: mobility-aware transmission scheduling, joint compute-communication optimization, and model compression frameworks.

A. Mobility-Aware Transmission Scheduling

Several studies focus on wireless channel uncertainty and vehicle mobility. Hong et al. [1] modeled Vehicle-to-Infrastructure (V2I) model uploads as a Markov Decision Process (MDP) to reduce latency and power consumption. Karatas et al. [2] proposed variable-rate client selection to mitigate imperfect channel state information. Yan et al. [3] introduced opportunistic Vehicle-to-Vehicle (V2V) relays to facilitate model transfers under energy constraints. These methods address link degradation but treat vehicular compute capacity as an unconstrained or static parameter.

B. Joint Compute and Communication Co-Design

To address the coupling between onboard execution time and wireless transmission delay, joint resource allocation frameworks have been explored. Qiang et al. [4] investigated dynamic model partitioning across vehicular edge nodes based on instantaneous processor and bandwidth availability. Ballotta et al. [5] proposed a predictive computation-scheduling method using radio environment maps. Zhang et al. [6] jointly optimized client selection and resource allocation to improve convergence accuracy, while Pervej et al. [7] formulated iteration counts and CPU frequencies alongside physical resource block allocations. Although these formulations link computation latency with transmission budgets, they generally assume dedicated hardware partitions for FL.

C. Compressed and Asynchronous Frameworks

To reduce communication bottlenecks and handle stragglers, quantization and asynchronous aggregation have been deployed. Liu et al. [8] optimized local batch sizes and gradient compression ratios under latency bounds. Zhang et al. [9] evaluated adaptive quantization schemes based on channel conditions. In asynchronous settings, Wu et al. [10] tuned upload frequency to instantaneous link quality, and Zhang et al. [11] deployed sliding-window parameter transfers on embedded automotive GPUs.

D. Summary and Positioning

As summarized in Table I, existing approaches primarily optimize radio resource blocks or static computation partitions. In contrast, modern SDVs operate in mixed-criticality environments where FL shares compute capacity with high-priority safety tasks. Our work introduces a dynamic compute occupancy metric ($U_i^{\text{crit}}(t)$) and implements an online heuristic that co-optimizes training depth and communication timing based on real-time vehicle resource constraints.

III. System Model and Problem Formulation

We consider a vehicular federated learning network with N client vehicles, indexed by $\mathcal{V} = \{1, 2, \ldots, N\}$, communicating with an edge or roadside server over discrete decision rounds $t \in \{0, 1, \ldots, T-1\}$.

A. Vehicle State Representation

At round t, each vehicle $i \in \mathcal{V}$ observes a local state vector:

$$S_i(t) = \{B_i(t),\, U_i^{\text{crit}}(t),\, D_i(t)\} \tag{1}$$

where:

- $B_i(t)$ represents the instantaneous uplink bandwidth available to vehicle i.
- $U_i^{\text{crit}}(t) \in [0, 1]$ denotes the fraction of onboard compute capacity occupied by safety-critical tasks (e.g., perception and trajectory planning).
- $D_i(t)$ indicates client model drift relative to the global model parameters.

B. Cost and Gain Formulation

The operational costs of participating in training round t are defined as follows:

1) Communication Cost: Let $p_i(t) \in \{0, 1\}$ denote the binary transmission decision ($p_i(t) = 1$ indicates model upload). The network transmission delay penalty is modeled as:

$$C_{\text{net}}(t) = \sum_{i \in \mathcal{V}} p_i(t) \frac{M_i}{B_i(t)} \tag{2}$$

where M_i is the parameter payload size in megabytes.

2) Compute Contention Cost: Let $c_i(t) \in [0, 1]$ represent the compute allocation granted to the FL process. The contention penalty imposed on co-located services is defined as:

$$C_{\text{comp}}(t) = \sum_{i \in \mathcal{V}} c_i(t) U_i^{\text{crit}}(t) \tag{3}$$

3) Model Divergence: We measure client drift $D_i(t)$ directly as the parameter Euclidean distance between local parameters $w_{\text{local},i}(t)$ and the received global parameters $w_{\text{global}}(t)$:

$$D_i(t) = \| w_{\text{local},i}(t) - w_{\text{global}}(t) \|_2 \tag{4}$$

The cumulative drift across the participating fleet is:

$$C_{\text{drift}}(t) = \sum_{i \in \mathcal{V}} D_i(t) \tag{5}$$

4) Learning Utility: To account for optimization progress, we track the local empirical loss reduction across iterations:

$$G_i(t) = L_i(t-1) - L_i(t) \tag{6}$$

The immediate net utility of continued local computation is evaluated as:

$$U_i(t) = \delta G_i(t) - \gamma D_i(t) - \beta c_i(t) U_i^{\text{crit}}(t) \tag{7}$$

where δ, γ, and β are non-negative weighting coefficients.

C. Optimization Problem

The global fleet objective over time horizon T is formulated as:

$$\max_{\{c_i(t), e_i(t), p_i(t)\}} \sum_{t=0}^{T-1} \left[\delta \sum_{i \in \mathcal{V}} G_i(t) - \alpha C_{\text{net}}(t) - \beta C_{\text{comp}}(t) - \gamma C_{\text{drift}}(t) \right] \tag{8}$$

subject to:

$$c_i(t) \le 1 - U_i^{\text{crit}}(t), \quad \forall i, t \tag{9}$$

$$e_i(t) \le \rho_i c_i(t), \quad \forall i, t \tag{10}$$

$$\sum_{i \in \mathcal{V}} p_i(t) M_i \le B_{\text{cell}}(t), \quad \forall t \tag{11}$$

$$D_i(t) \le D_i^{\max}, \quad \forall i, t \tag{12}$$

Constraint (9) restricts FL execution to available compute slack. Constraint (10) bounds local training epochs $e_i(t)$ by available compute via scaling factor ρ_i. Constraint (11) limits simultaneous uploads to total cell uplink capacity $B_{\text{cell}}(t)$, and (12) bounds client parameter deviation.

TABLE I
COMPARISON OF VEHICULAR FL RESOURCE ALLOCATION APPROACHES

Reference	Compute Modeling	Network Modeling	Optimization Objective
Hong et al. [1]	Implicit / fixed execution latency	Uplink transmit power and duration	Minimize weighted upload delay and energy
Karatas et al. [2]	Uniform client processing rate	Imperfect CSI, adaptive transmission rate	Accelerate convergence under link uncertainty
Yan et al. [3]	Static local compute baseline	V2V/V2I relay and opportunistic links	Maximize upload success probability
Qiang et al. [4]	CPU frequency and split layers	Dynamic channel bandwidth	Minimize total processing and transmission delay
Ballotta et al. [5]	Local iteration scheduling	Radio environment maps, path loss	Optimize training progress under round deadlines
Zhang et al. [6]	General compute resource bounds	Channel-aware client selection	Maximize final model accuracy
Pervej et al. [7]	CPU frequency and iteration bounds	PRB allocation, sojourn time	Joint energy-delay trade-off under mobility
Liu et al. [8]	Batch size scaling	Gradient compression ratio, spectrum allocation	Maximize convergence rate under delay caps
Zhang et al. [9]	Latency-coupled computation	Multi-level parameter quantization	Trade off quantization error and latency
Wu et al. [10]	Adaptive local training loops	Asynchronous upload scheduling	Minimize transmission redundancy
Zhang et al. [11]	Embedded GPU hardware evaluation	Sliding-window parameter exchange	Asynchronous convergence on embedded systems
Proposed	**Dynamic safety workload occupancy (U_i^{crit})**	**Time-varying cell capacity and throughput**	**Jointly minimize bandwidth, model drift, and compute contention**

Because future vehicle trajectories, channel throughput, and safety workloads are non-deterministic, solving this optimization problem offline is intractable. Consequently, we implement an online deterministic policy engine that approximates this objective at each decision interval.

IV. RUNTIME POLICY ENGINE

At each decision interval t, the client evaluates its local state and transitions among three discrete execution states:

- **WAIT (Idle):** If available compute slack is insufficient for a complete training epoch ($1 - U_i^{\text{crit}}(t) < 1/\rho_i$), the client skips training ($e_i = 0, p_i = 0$) to avoid compute contention.
- **CONTINUE (Local Training):** When compute slack permits, the vehicle trains locally. Training is bounded using an exponential loss fit $L(e) \approx L_\infty + Ae^{-Be}$ to predict marginal gains. The local loop terminates when expected gain falls below threshold G_{thresh}, drift exceeds $D_i^{\max}$, or local utility drops below τ_{continue}.
- **SYNC (Upload):** Upon concluding local training, the client checks bandwidth $B_i(t)$. If $B_i(t) \geq B_{\text{thresh}}$, the client uploads its update ($p_i = 1$). If the channel is constrained, the client retains weights locally and transitions back to CONTINUE in the subsequent round, preventing channel drops.

V. PERFORMANCE EVALUATION

A. Experimental Setup

The evaluation testbed is implemented using PyTorch and the Flower federated learning framework. The client workload uses a ResNet-9 architecture modified with Group Normalization (8 groups) across convolutional stages to mitigate running-statistic errors under non-IID data partitioning. Training is conducted on the CIFAR-10 dataset distributed over $N = 10$ vehicular clients using a Dirichlet partition ($\alpha_{\text{Dir}} = 0.2$).

Baseline comparisons include fixed local training regimes of 1, 3, 5, 10, and 20 epochs. The dynamic policy operates with tuned coefficients: $\alpha = 0.005$, $\beta = 1.0$, $\gamma = 0.006$, and $\delta = 27.0$. Cellular uplink capacity follows a dynamic envelope to evaluate behavior under fluctuating channel availability.

B. Convergence and Network Efficiency

As shown in Fig. **??**, the dynamic policy achieves convergence comparable to high-frequency synchronization configurations (Fixed 1 and 3 epochs) over 250 communication rounds. Fixed configurations with high local epochs (10 and 20) exhibit slower convergence rates due to client drift under non-IID data.

Fig. 2 illustrates cumulative network bandwidth consumption relative to validation loss. The dynamic policy reaches its terminal loss using under 2,000 MB of transmitted data. In contrast, static low-epoch baselines require over 7,500 MB to attain equivalent validation metrics, representing an uplink bandwidth reduction of over 70%.

C. Compute Contention and State Distribution

Fig. 3 presents the cumulative progression of utility, bandwidth, compute interference, and model drift. High-epoch configurations (Fixed 10 and 20) accumulate severe compute interference by executing local training regardless of safety task occupancy. The proposed policy throttles training during periods of high safety-critical load, maintaining cumulative interference within acceptable bounds while keeping drift aligned with 3-epoch baselines.

The mechanism behind these results is illustrated in Fig. 4. Fixed-epoch strategies transmit parameters without inspecting channel throughput, resulting in numerous dropped updates

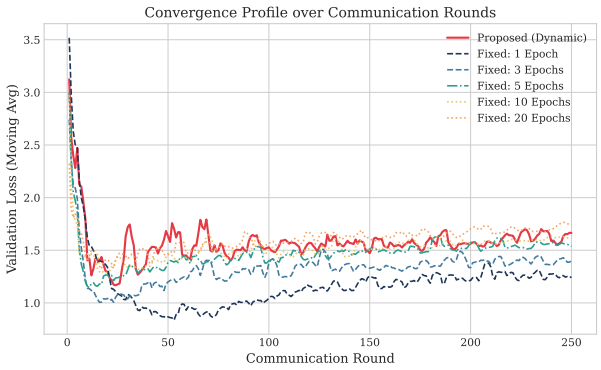


Fig. 1. Validation loss convergence across communication rounds for dynamic and fixed-epoch baselines.

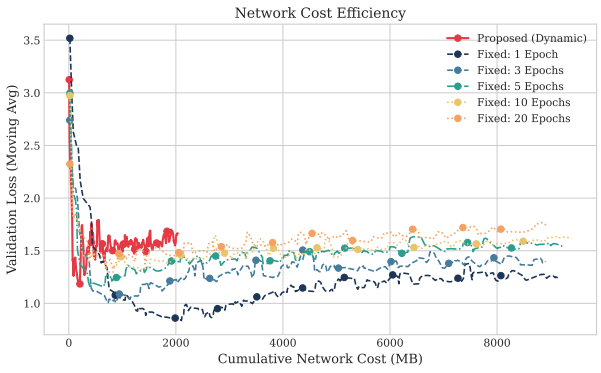


Fig. 2. Validation loss as a function of cumulative uplink network consumption (MB).

when aggregate transmissions exceed cell capacity (B_{cell}). The dynamic engine avoids buffer drops by routing clients into CONTINUE or WAIT states during poor channel conditions, synchronizing only when link throughput supports transmission.

VI. Conclusion

This paper presented a runtime policy engine for federated learning in Software-Defined Vehicles that addresses compute contention and V2X bandwidth constraints. By treating in-vehicle training as a secondary workload and dynamically regulating local epochs and upload timing, the proposed engine avoids uplink congestion while preserving safety-critical compute slack. Experimental results on non-IID benchmarks show that the policy maintains global model convergence while reducing cumulative transmission volume by more than 70% relative to static scheduling. Future work will investigate integrating mobility prediction models to proactively adjust local batch execution prior to wireless coverage dropouts.

References

[1] Y. Hong, B. Lv, S. Wang, R. Wang, H. Tan, and F. C. Lau, "Fedcars: An efficient scheduling framework for in-vehicle federated learning integrated with carla-cosimulation," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 5, pp. 7997–8011, 2025.

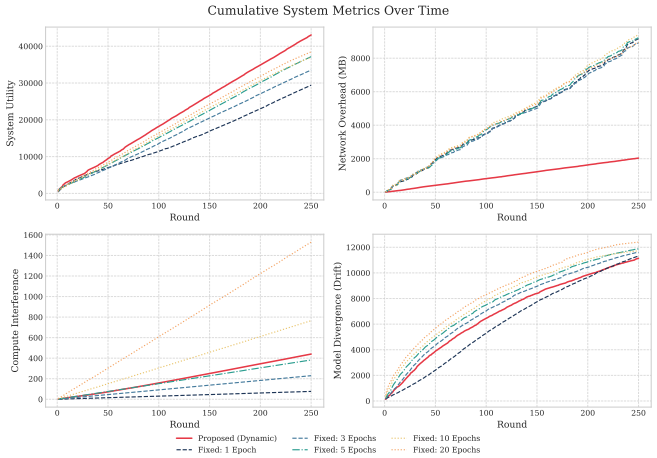


Fig. 3. Cumulative system metrics over 250 rounds: utility, network consumption, compute interference, and model drift.

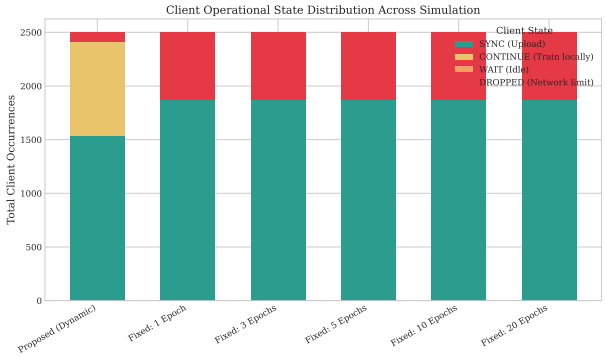


Fig. 4. Distribution of client operational states across the 250-round simulation.

[2] M. Karatas et al., "VR-VFL: Joint rate and client selection for vehicular federated learning under imperfect CSI," *IEEE Transactions on Mobile Computing*, early access, 2026.

[3] J. Yan, T. Chen, Y. Sun, Z. Nan, S. Zhou, and Z. Niu, "Dynamic scheduling for vehicle-to-vehicle communications enhanced federated learning," *IEEE Transactions on Wireless Communications*, pp. 9373–9390, 2024.

[4] X. Qiang, Z. Chang, Y. Hu, L. Liu, and T. Hämäläinen, "Adaptive and parallel split federated learning in vehicular edge computing," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4591–4604, 2025.

[5] L. Ballotta, N. D. Fabbro, G. Perin, L. Schenato, M. Rossi, and G. Piro, "VREM-FL: Mobility-aware computation-scheduling co-design for vehicular federated learning," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3311–3326, 2025.

[6] X. Zhang, Z. Chang, T. Hu, W. Chen, X. Zhang, and G. Min, "Vehicle selection and resource allocation for federated learning-assisted vehicular network," *IEEE Transactions on Mobile Computing*, vol. 23, no. 5, pp. 3817–3829, 2024.

[7] M. F. Pervej, R. Jin, and H. Dai, "Resource constrained vehicular edge federated learning with highly mobile connected vehicles," *IEEE Journal on Selected Areas in Communications*, pp. 1825–1844, 2023.

[8] S. Liu, G. Yu, R. Yin, J. Yuan, and F. Qu, "Communication and computation efficient federated learning for internet of vehicles with a constrained latency," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 1, pp. 1038–1052, 2024.

[9] X. Zhang, W. Chen, H. Zhao, Z. Chang, and Z. Han, "Joint accuracy and latency optimization for quantized federated learning in vehicular networks," *IEEE Internet of Things Journal*, vol. 11, no. 17, pp. 28876–28890, 2024.

[10] M. Wu, M. Boban, and F. Dressler, "Flexible training and uploading strategy for asynchronous federated learning in dynamic environments," *IEEE Transactions on Mobile Computing*, pp. 12907–12921, 2024.

[11] H. Zhang, J. Bosch, and H. H. Olsson, "Real-time end-to-end federated learning: An automotive case study," in *Proc. IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*, 2021, pp. 459–468.